

App Dev Project Report

1. Student Details

Name: Shakthi Kumaran Gnanavel

Roll Number: 23f2003594

Email: 23f2003594@ds.study.iitm.ac.in

About Me: I am a student currently pursuing Diploma in the IIT Madras BS Degree Program with an interest in machine learning and web development , particularly in designing and building scalable , user-centric applications . I enjoy working across the full stack , integrating frontend and backend systems to create efficient and reliable solutions .

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To design and develop a web-based placement management system that streamlines the campus recruitment process by connecting students, companies, and administrators on a single platform. The system aims to manage job postings, applications, candidate shortlisting, interview scheduling, and final placement tracking efficiently.

Approach:

The application was developed using Flask as the backend framework with a modular architecture separating authentication, admin, student, and company functionalities. The frontend was built using Vue.js to create a responsive and dynamic user interface, while Axios was used for API communication between frontend and backend.

The system enables role-based access where students can apply for jobs and track their application status, companies can manage recruitment workflows, and administrators oversee approvals and system integrity. Data is stored using SQLite, and session-based authentication ensures secure access. The platform also supports real-time status updates, interview scheduling, and offer management using external file links

3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** to assist in building parts of the system from the ground up, particularly in understanding and structuring the Express.js middleware API layer and developing the Vue.js interface. AI was also used for generating API documentation (OpenAPI/YAML), refining backend route structures, and improving code organization and naming consistency. Additionally, **Gemini AI 3** was used to enhance Vue.js components and UI responsiveness.

The extent of AI/LLM usage is estimated to be around **15–18%**, mainly for initial scaffolding, code suggestions, and documentation support. All final implementation logic, debugging, authentication flow (JWT), API integration, and system design decisions were carried out manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Vue.js	Frontend framework for building the reactive User Interface.
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
Axios	Promise-based HTTP client used for frontend-middleware-backend communication.
Express.js	Middleware gateway for handling API routing , authentication , and communication with Flask
JSON Web Token	Secure token-based authentication for role-based access control.
SQLite	Lightweight local database for storing user data
Flask-CORS	Extension to handle Cross-Origin Resource Sharing across the 3-tier setup.
Werkzeug	Utility library used for secure password hashing and verification.

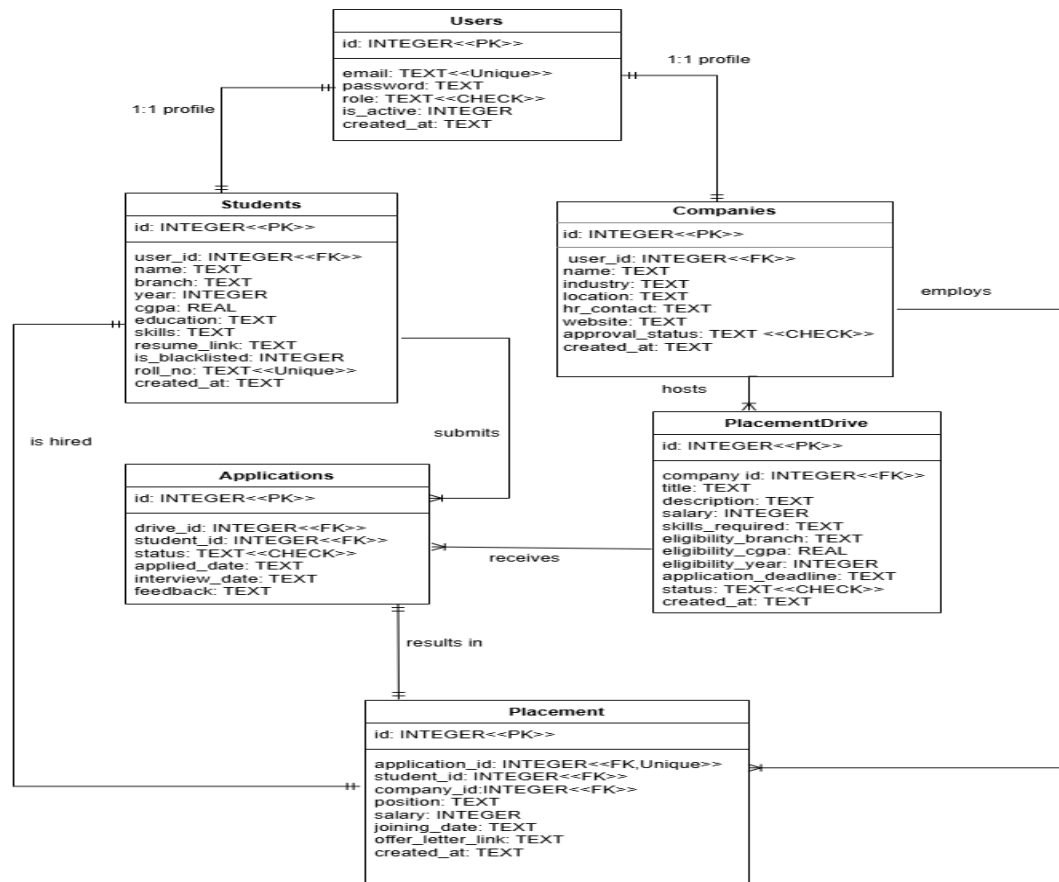
5. Database Schema / ER Diagram

Tables:

1. **Users** — The primary authentication table that stores credentials and access levels for everyone using the system. (id, email, password, role, is_active, created_at)
2. **companies**— Stores profile information of recruiting organizations. (id, user_id, name, industry, location, hr_contact, website, approval_status, created_at)
3. **students** — Stores academic and personal data for candidates. (id, user_id, name, branch, year, cgpa, education, skills, resume_link, created_at, is_blacklisted, roll_no)
4. **placement_drive** — Represents specific recruitment events or job openings created by a company. (id, company_id, title, description, salary, skills_required, eligibility_branch, eligibility_cgpa, eligibility_year, application_deadline, status, created_at)
5. **applications** — A junction table managing the Many-to-Many relationship between students. (id, drive_id, student_id, status, applied_date, interview_date, feedback)
6. **placement** — The "Success" table. Records finalized hiring details once an offer is accepted. (id, application_id, student_id, company_id, position, salary, joining_date, created_at, offer_letter_link)

Relationships

- One-to-One → Users → Students
- One-to-One → Users → Companies
- One-to-One → Applications → Placement
- One-to-Many → Companies → Placement_Drive
- One-to-Many → Placement_Drive → Applications
- One-to-Many → Students → Applications
- One-to-Many → Companies → Placement
- One-to-Many → Students → Placement
- Many-to-Many → Students ↔ Placement_Drive (via Applications)



6. API Resource Endpoints

Endpoint	Method	Description
/api/test	GET	Tests is Express server is running
/api/login	POST	Authenticates user and initializes role-based session.
/api/register/student	POST	Creates a new student profile in the database.
/api/register/company	POST	Submits company details for Admin verification.
/api/student/dashboard	GET	Displays application status and interview alerts.

<code>/api/student/drives</code>	GET	Fetches job drives
<code>/api/student/apply</code>	POST	Processes a new job application submission.
<code>/api/student/profile</code>	POST	Updates student academic and professional details.
<code>/api/student/profile</code>	GET	Retrieves student profile
<code>/api/student/applications</code>	GET	Retrieves student applications
<code>/api/student/applications/{userId}</code>	GET	Retrieves applications with access validation
<code>/api/student/accept_offer</code>	POST	Accepts an job offer and marks the placement.
<code>/api/student/reject_offer</code>	POST	Rejects an offer
<code>/api/company/stats</code>	GET	Retrieves company specific statistics
<code>/api/student/reject_offer/{id}</code>	GET	Formally declines a pending job offer.
<code>/api/company/drives</code>	GET	Retrieves drives created by the company
<code>/api/company/drives/create</code>	POST	Create a new placement drive
<code>/api/company/drives/{id}/applications</code>	GET	Retrieves applicants for a drive
<code>/api/company/applications/{id}/shortlist</code>	POST	Shortlists a candidate
<code>/api/company/applications/{id}/reject</code>	POST	Rejects a candidate
<code>/api/company/applications/{id}/schedule</code>	POST	Schedules interview and adds feedback

<code>/api/company/drives/{id}</code>	GET	Retrieves selected candidates
<code>/api/company/applications/{id}/select</code>	POST	Selects candidate and issues offer
<code>/api/company/drives/{id}/selected</code>	GET	Retrieves selected candidates
<code>/api/company/students/{id}</code>	GET	Fetches student profile for review
<code>/api/admin/dashboard</code>	GET	Fetches overall platform statistics.
<code>/api/admin/companies</code>	GET	Retrieves list of all companies
<code>/api/admin/companies/pending</code>	GET	Retrieves companies awaiting approval
<code>/api/admin/company/{id}/approve</code>	POST	Approves a company
<code>/api/admin/company/{id}/reject</code>	POST	Rejects a company
<code>/api/admin/companies/{userId}/toggle-active</code>	POST	Activates / Deactivate a company
<code>/api/admin/students/{id}/toggle-blacklist</code>	POST	Blacklists/Unblacklists a student
<code>/api/admin/drives</code>	GET	Retrieves all placement drives
<code>/api/admin/drives/{id}/action</code>	POST	Performs action (approve/close) on a drive
<code>/api/admin/drives/{id}/applications</code>	GET	Retrieves applications for a drive
<code>/api/admin/placements</code>	GET	Retrieves all final placements
<code>/api/admin/students</code>	GET	Retrieves list of all students

`/api/admin/students/{` GET
`id}`

Fetches a student profile

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1PNDj5qerS8be-_jFmlxrQPukl-fYfIC0/view?usp=sharing
